

Rapport V2 – SonicWatch

Ali Can CEBI Anisse HAMDI Paul RABIN
Lucas LEFEVBRE

14 décembre 2025

Table des matières

1	Introduction	2
2	Installation et mode d'emploi	2
2.1	Pré-requis	2
2.2	Démarrage rapide	2
2.3	Mode manuel	3
3	Méthodologie	3
3.1	LLM et prompt	3
3.2	Modèle spécialiste	3
3.3	MCP	4
3.4	Interface utilisateur	4
4	Développement : difficultés et solutions	4
5	Conclusion	5

1 Introduction

SonicWatch est un assistant audio qui explique les sorties d'un classifieur de sons urbains. Le modèle spécialiste est entraîné sur UrbanSound8K. L'utilisateur fournit un fichier WAV, le chatbot déclenche l'inférence puis commente les résultats en français.

Afin de concilier robustesse et flexibilité, nous séparons les rôles : un LLM généraliste (LM Studio + Mistral) assure la couche conversationnelle, un modèle PyTorch calcule les probabilités, un serveur MCP (FastAPI) sert d'intermédiaire et expose les métriques, et deux orchestrateurs (CLI et Gradio) pilotent l'expérience utilisateur.

2 Installation et mode d'emploi

2.1 Pré-requis

Avant de lancer le projet, il faut :

- Python 3.8 à 3.12
- LM Studio installé sur votre PC
- Le checkpoint pré-entraîné `weights/urbansound_cnn.pt`, fourni avec l'archive

Après décompression de l'archive :

```
cd DeepLearningProject
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

2.2 Démarrage rapide

Depuis la racine du dépôt, exécuter :

```
bash setup.sh
```

Ce script détecte ou crée l'environnement virtuel, installe les dépendances, vérifie la présence des poids pré-entraînés puis lance le serveur MCP et l'interface Gradio.

Ensuite :

1. Ouvrir LM Studio, charger un modèle Instruct (ex. `mistralai/mistral-7b-instruct`) et lancer le serveur local sur `http://127.0.0.1:1234`
2. Ouvrir l'URL Gradio affichée dans le terminal (`http://127.0.0.1:7860`)

2.3 Mode manuel

Pour lancer manuellement les services :

- **Serveur MCP** : `uvicorn mcp_server.server:app --host 127.0.0.1 --port 8000`
- **Serveur LLM** : ouvrir LM Studio et démarrer le serveur
- **Interface Web** : `python app_gradio.py`

L'utilisateur peut saisir une consigne textuelle ou importer un WAV via l'interface graphique.

3 Méthodologie

3.1 LLM et prompt

LM Studio sert de passerelle compatible OpenAI. Le prompt système (chatbot/prompt.md) définit SonicWatch. Les orchestrateurs utilisent `openai.OpenAI` avec `base_url` pointant vers LM Studio. Le LLM ne voit jamais les WAV : il ne reçoit que des résumés JSON issus du serveur MCP.

3.2 Modèle spécialiste

UrbanSound8K contient ~8732 extraits répartis en 10 classes et 10 folds. Dans `model/` :

- `train.py` : pipeline principal (mel-spectrogrammes, CNN PyTorch, SpecAugment, WeightedRandomSampler)
- `precompute_embeddings.py` et `train_embeddings.py` : pipeline base-line PANNs

Deux variantes sont maintenues :

1. **CNN from scratch** : 217k paramètres, exporté dans `weights/urbansound_cnn.pt`. Atteint **71.3% d'accuracy** et 72.5% de macro-F1.

2. **Baseline PANNs** : projection via PANNs (transfer learning) + classifieur LogReg. Atteint **84.3% d'accuracy**, confirmant l'efficacité du pré-entraînement.

Les métriques sont exportées dans `reports/metrics*.json` et des matrices de confusion PNG.

3.3 MCP

`mcp_server/server.py` est une API FastAPI qui charge les poids au démarrage et expose :

- `GET /health` : statut, device, checkpoint utilisé
- `POST /infer` : renvoie `predicted_label`, `confidence`, `is_confident`, `top_k`
- `GET /metrics?variant={cnn,embeddings}` et `GET /reports`

3.4 Interface utilisateur

`app_gradio.py` fournit l'UI Web basée sur `gr.Blocks` avec un layout en deux colonnes (chat à gauche, upload à droite). Elle inclut un en-tête glassmorphism, des exemples cliquables et un module d'upload WAV (`gr.File`).

4 Développement : difficultés et solutions

- **Mise en place de l'environnement** : UrbanSound8K est volumineux. Nous avons scripté `setup.sh` qui télécharge le dataset si nécessaire et lance le serveur MCP avec les poids pré-entraînés.
- **Conception du modèle** : pour équilibrer les classes et limiter l'overfitting, nous combinons `WeightedRandomSampler`, `SpecAugment` et `early stopping`. La baseline PANNs sert de garde-fou.
- **Intégration LLM** : LM Studio n'accepte que les rôles `user/assistant`. Nous transformons donc les prompts système via un helper `build_messages`.
- **Interface Gradio** : nous avons remplacé le composant `Audio` par `gr.File` et résolu un bug de compatibilité Python 3.12/Gradio.

dio 4.44 (TypeError: 'bool' is not iterable) en patchant `gradio_client.utils`.

5 Conclusion

Cette **deuxième version** de SonicWatch consolide le projet avec plusieurs améliorations :

Ce qui a été accompli :

- Un CNN fonctionnel atteignant 71.3% d'accuracy avec 217k paramètres
- Une baseline PANNs atteignant 84.3% d'accuracy
- Une interface Gradio moderne avec chat conversationnel
- Une architecture micro-services modulaire
- Résolution de bugs de compatibilité Python 3.12 / Gradio 4.44

Perspectives d'amélioration :

1. Fine-tuning de PANNs pour dépasser les 84% actuels
2. Collecte de données pour les classes difficiles (`children_playing`, `street_music`)
3. Conteneurisation Docker pour un déploiement simplifié
4. Enregistrement micro en temps réel